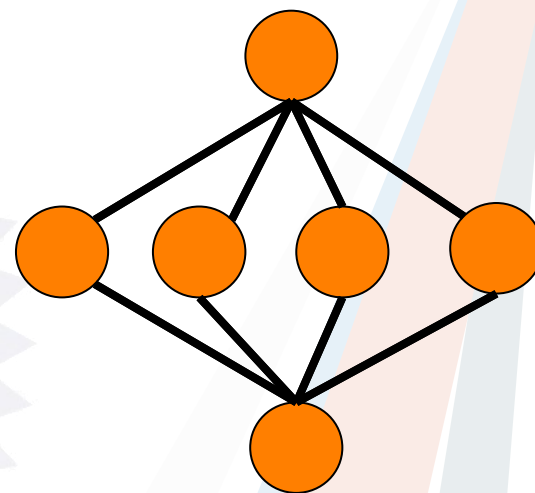
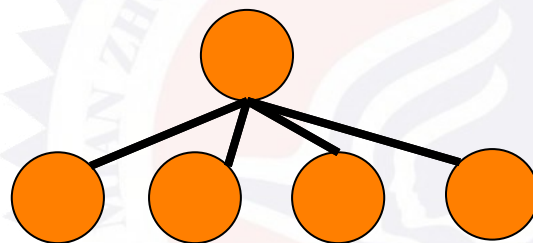


线性数据结构

主讲：黄凤林

数据结构

维护数据与数据之间的关系的存储结构



线性类数据结构:

数组: 任意位置访问存储

栈: 一端操作 (查询、插入、删除等)

队列: 两端操作 (队尾进、队首出)

链表: 每次查询从头开始, 插入和删除比较方便

线性数据结构

数组：物理位置相邻，逻辑相邻

栈：特殊数组，一端操作(栈顶)

队列：特殊数组，两端操作(队首队尾)

链表：逻辑相邻，物理不相邻，只能从链头操作

分块：一种优雅的暴力数据结构(当你还不是大佬的时候很实用，让你得到意想不到的分数)

哈希表：常常用于状态判重



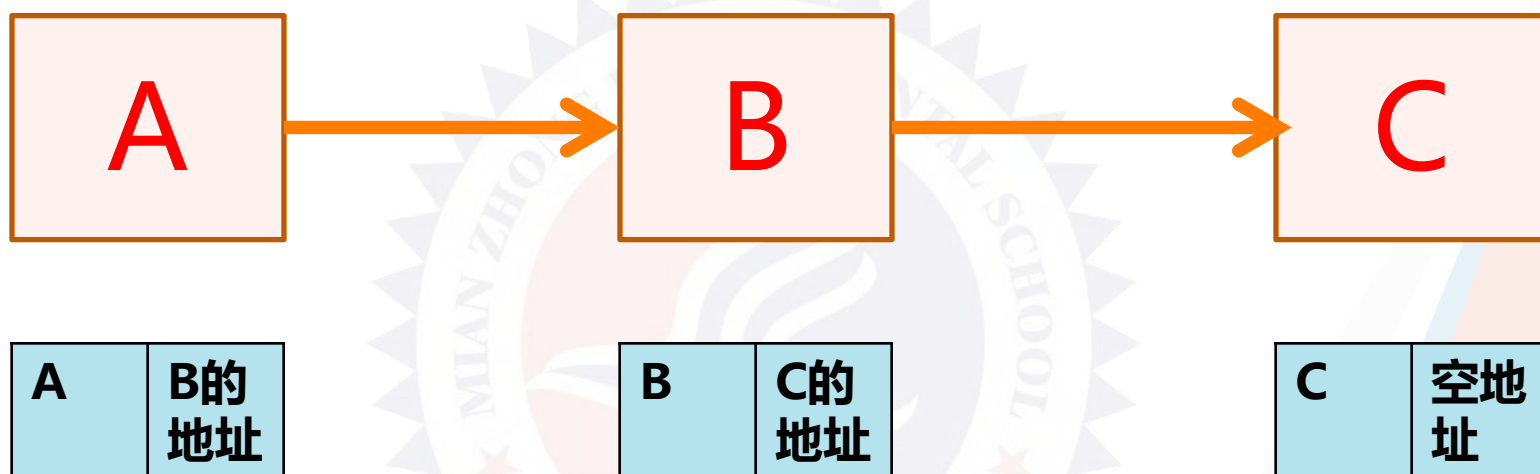
➔ 数组模拟练习题目

- 1: 洛谷 1092 合并果子
- 2: 洛谷 2827 蚯蚓
- 3: 洛谷 1969 积木大赛
- 4: MSOJ 1382 信箱
- 5: MSOJ 1377 排列交换



→ 链表

链表：物理上独立，逻辑上相邻的线性表



链表的实现：结构体指针和数组模拟指针



➔ 指针

指针（在普通变量前加个*）

指针：指向存储地址的变量

指针变量：专门用来存放变量地址的变量就称指针变量。

当指针变量中存放着另一个变量的地址时，就称这个指针变量指向那一变量。

注：指针变量在使用前必须先申请空间



→ 指针举例

指针变量的定义:

`int *p;` // `p` 就是一个可以存放其他变量地址的地址变量。

`int q;` // 值变量

`int main()`

{

`p = new (int);` // 申请指针变量空间

`scanf("%d",&q);`

`p = &q;`

`printf("%d %d",q,*p);`

`return 0;`

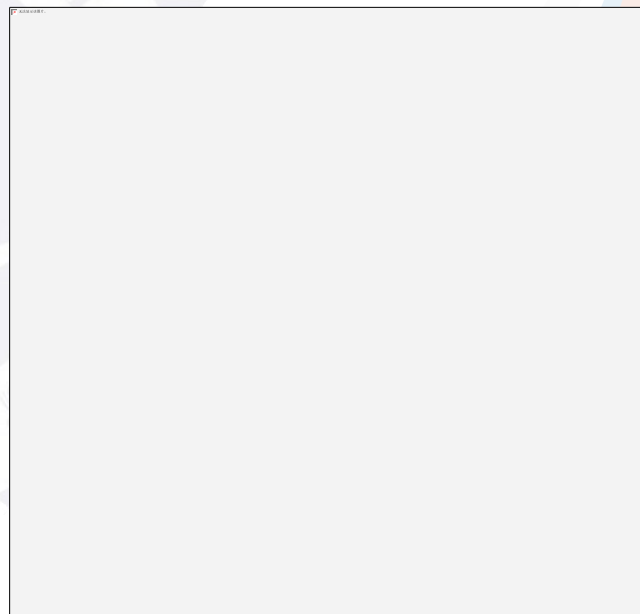
}



→ 指针

- 指针类型：指向内存地址的类型，用 **"*"** 表示，一般与常规变量一起使用。
- 指针和指针变量：定义一个变量就意味着在内存中给它分配一定的存储单元，以备存储这个变量的数值。

指针变量是一种特殊的变量，它在内存中保存的不是一般的数值，而是另一个变量的地址。



→ 指针变量说明

- 像其它任何一种变量一样，在C语言中使用指针变量之前，也必须先用说明语句定义指针变量。
- 指针变量定义语句的一般格式为：**数据类型 *变量名;**
- 符号 “*” 是定义指针变量的标志。

指针变量的使用：

- 1、指针变量名中的变量名存储的是指向一个变量的**地址**
- 2、如果要使用指针变量，**必须先给指针变量分配存储空间**，然后才能存储，否则会报不可预知的错误。
- 3、“*” 在定义时表示**指针变量标志**，“*” 在使用时，**变量名前面加个***，表示指针变量里面装的值。



→ 指针

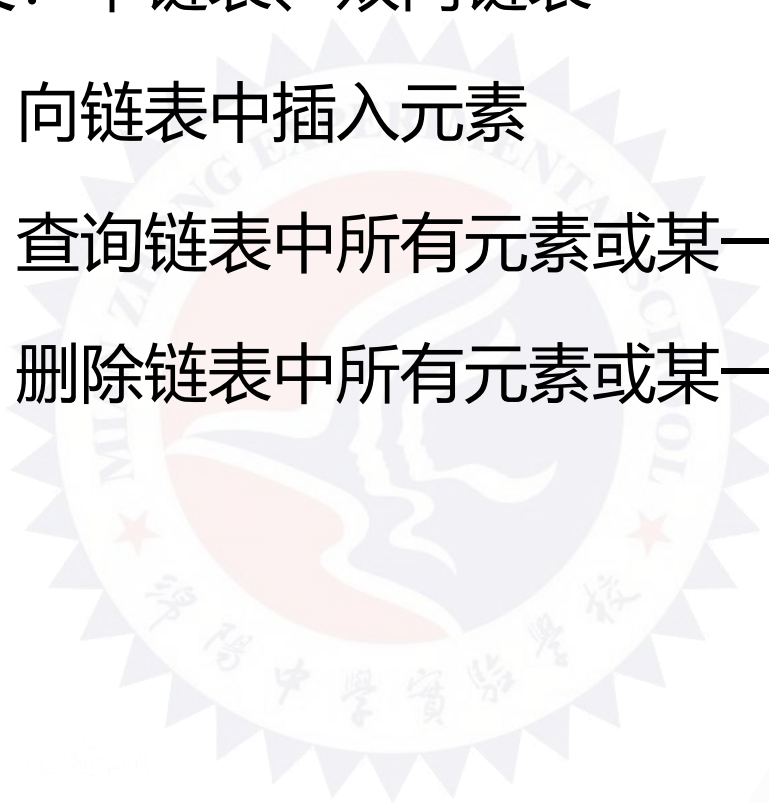
```
int *a;  
int main()  
{  
    a= new int; // 给指针变量分配存储空间  
    scanf("%d",a);  
    printf("%d",*a);  
    return 0;  
}
```





链表的操作

- 1、链表的定义：单链表、双向链表
- 2、插入操作：向链表中插入元素
- 3、查询元素：查询链表中所有元素或某一特定元素
- 4、删除元素：删除链表中所有元素或某一特定元素



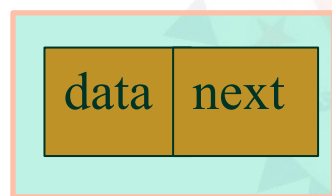
→ 链表---结构体指针实现

链表结点定义

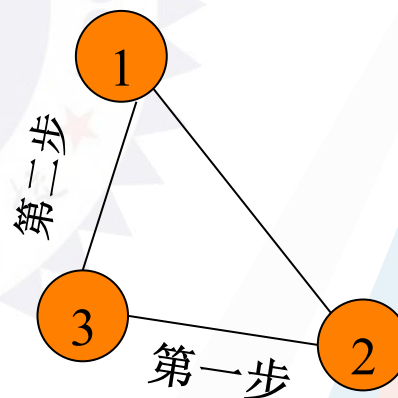
```
struct node{  
    char ch;  
    node *next;  
};  
node *head,*p;
```

插入操作:

```
p= new (node);  
p->ch=getchar();  
p->next=head->next;  
head->next=p;
```



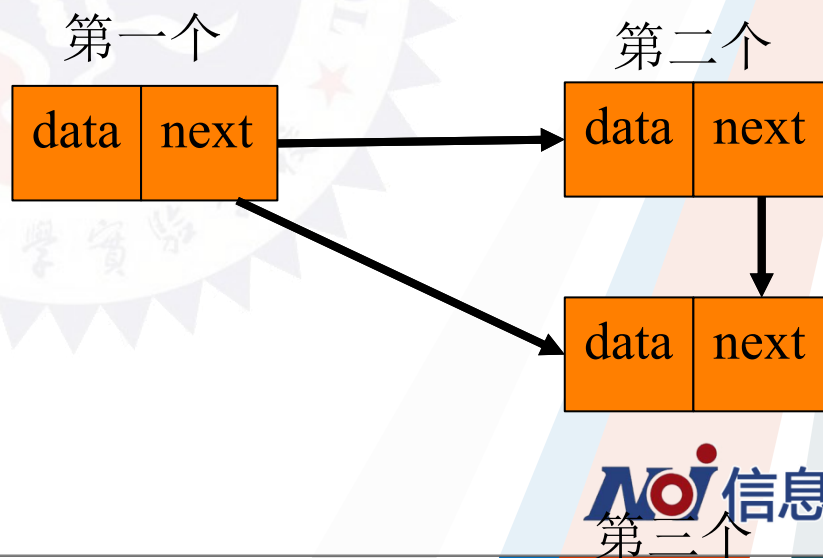
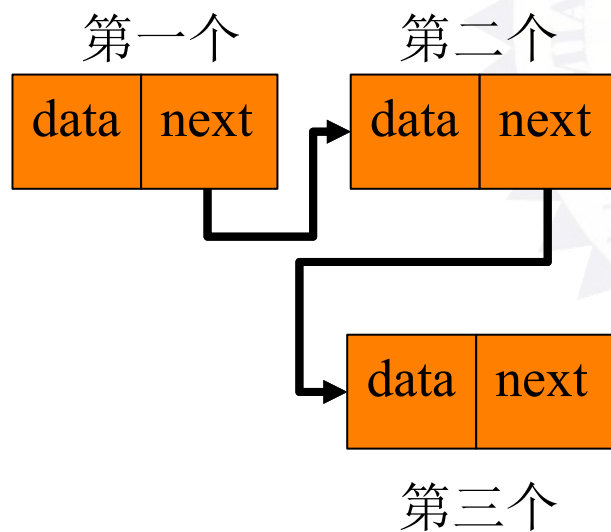
↑
head



链表---结构体指针实现

```
void select()
{
    p=new (node);p=head->next;
    while(p!=NULL)
    {
        printf("%c ",p->data);
        p=p->next;
    }
}
```

```
bool del(char ch){
    p=new (node); q=new (node);
    q=head;p=head->next;
    while(p!=NULL){
        if(p->data==ch){
            q->next=p->next;
            return true;
        }
        q=p; p=p->next;
    }
    return false;
}
```



链表---数组模拟实现

思想原理：利用数组下标来代替指针指向的地址

链表结点定义：

```
struct node{  
    int num;//元素的值  
    int next;//下一个结点在数组中的位置  
} list[maxn+maxm];  
int head;
```



head=1

1	2	3
1	3	5
2	3	0



链表操作

```
void insert(int val){//插入元素val后, 保证序列依然从小到大有序
    list[++k].num=val;list[k].next=0;
    if(head==0) {head=k; return;}
    if(list[head].num>val){
        list[k].next=head;
        head=k; return;
    }
    int p=head,q;
    while(p!=0){
        q=list[p].next;
        if(list[p].num<val && (val<list[q].num) || q==0){
            list[k].next=q;
            list[p].next=k;
            return;
        }
        p=list[p].next;
    }
}
```

```
void del(int val){//删除指定的元素val
    int p=head,q;
    if(list[head].num==val) { head=list[p].next; return; }
    while(p!=0){
        q=list[p].next;
        if(list[q].num==val){
            list[p].next=list[q].next; return;
        }
        p=list[p].next;
    }
}
```



链表的扩展

(1)双向链表:

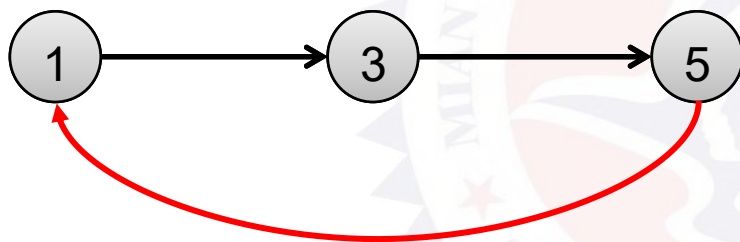
因为链表在物理存储上是无序的，所以不能直接通过下标找到当前元素的前一个或者后一个元素，一般都是在链表中的每一个元素中存储下一个元素的位置。一般的链表只有next指针，也就是只能通过上一个元素找到下一个元素，不能通过下一个元素找到上一个元素。如果想要正反都可以查找的链表，则可以用双向链表。即只需要再添加一个pre指针就可以了。

```
struct node{  
    int num; //值  
    int next; //下一个结点在数组中的位置  
    int pre; //下一个结点在数组中的位置  
};
```


链表的扩展

(2) 循环链表:

在数组中, 如果数组数循环的, 那么只需要用 $i \% n$ 就可以了(下标 $0 \sim n-1$). 但是在链表中明显不可以这样做, 并且在链表中很多时候我们不知道是否到链表的末尾了, 我们只需要让链表末尾指针指向首元素就可以了。



`list[n].next=head;`



链表的应用--约瑟夫问题

n 个人($n \leq 5000$)围成一圈,从第一个人开始报数,数到 m ($m \leq 5000$)的人出列,再由下一个人重新从1开始报数,数到 m 的人再出圈,.....依次类推,直到所有的人都出圈,请输出最后一个在圈里的人。

这道题看上去很简单,跟着答案模拟就可以了,但是实际操作过程中,你会发现,慢,特别慢,假设最后圈里面还有几个人,然后你要向前数5000,但是实际上你会循环的数很多圈,效率很低,所以我们可以考虑用链表。即用一个指针从当前位置指向下一个位置,模拟的时候只从有效位置跳,对于无效的位置直接pass就可以了。

MSOJ 1490 约瑟夫问题



链表的应用--文本编辑器

现在你要编写一个文本编辑器，该编辑器支持以下4个操作：

- 1、L:光标左移一格 (若光标已在最左边就无视这次操作)
- 2、R:光标右移一格 (若光标已在最右边就无视这条操作)
- 3、a-z:在光标处插入一个字母
- 4、B:删除光标前的一个字母 (等同Backspace键，若光标已在最左边就无视这次操作)

初始文本为空，现在给你一个操作的序列S，求操作结束后文本编辑器里面的内容。

输入样例:

bLuLaLa

输出样例:

aaub

输入样例:

BBLLLaRRRRBb

输出样例:

b



→ 链表与数组的区别

	顺序表	链表
访问	直接访问 $O(1)$	从头遍历 $O(n)$
插入	插入位置之后元素都发生变化 $O(n)$	只修要修改插入位置前驱后继 $O(1)$
删除	同插入	同插入

对于普通线性结构，如果插入删除操作远多于查询操作，可以考虑链表；如果访问操作远多于插入删除操作，最好使用顺序表

→ 栈 (S T A C K)

栈 (S t a c k) : 一种后入先出 (L I F O) 的数据结构。

操作要求: 只允许在末端 (一端) 进行删除和插入操作

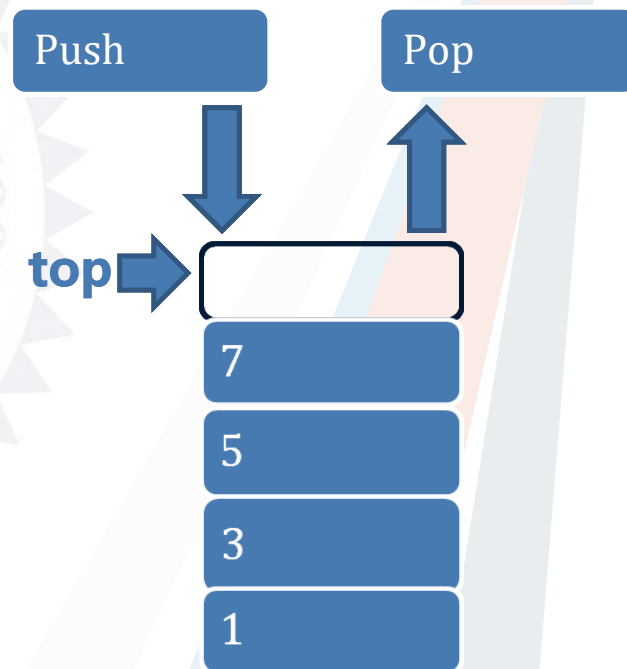
基本操作:

入栈(Push): 往栈顶添加新的元素

出栈(Pop): 删除栈顶元素。

相较于普通的数组, 栈有什么不同?

栈顶指针





数组模拟栈

栈的定义：

先定义一个数组：`int stack[100];`

再定义一个栈顶指针，即栈顶元素的位置：`int top=0;`
`top`表示栈顶元素的位置。**`top=0`表示栈为空**

入栈 (Push)：往栈顶添加元素 X

`top++;`

`stack[top]=X;`

出栈 (Pop)：弹出栈顶元素并且赋值给变量 X

`X=stack[top];`

`top--;`

判断栈是否为空：

`if (top==0) ...;`

栈的有限长度就是从
起点到栈顶的位置

→ STL 栈(C++)

在C++的STL函数库中，自带一些基本的数据结构。其中就包括我们今天学习的栈，我们可以直接调用

头文件：`#include<stack>`

定义：`stack<int> s;`

取栈顶元素：`s.top()`;

入栈(压栈)：`s.push(m)`;

出栈(弹栈)：`s.pop()`;

判断栈是否为空：`s.empty()`;

栈中元素有效个数：`s.size()`;

定义一个数据类型为int的栈 s，其中存储空间自动增长，不需要设置

入栈和出栈操作时，不需要更改栈顶元素的位置，因为他会自动更新

如果栈为空，返回1，
如果栈不为空，返回0



栈的应用

- 1: 栈模拟(多用于字符串操作)
- 2: 单调栈
- 3: 卡特兰数
- 4: 括号匹配, 表达式求值

NOI



→ 小W的爱情密码

题目描述

小W终于鼓起勇气向小 M 表白，然而只是有勇气写情书。为了防止情书内容被同学窃取，小W给情书加密。小 M 的解密方式很简单，假设情书是字符串 S1，小 W 给她的解密串是 S2,小 M 会重复地完成“在S1 中找到子串 S2 并删除”这一操作直到在 S1 中找不到 S2。

假如你是小 M，请你确定情书的最终内容。

数据范围

$\text{Length}(S1) \leq 10^6$

$\text{Length}(S2) \leq 100$

字符都是小写字母且无空格。

→ 小W的爱情密码—问题分析

问题分析

从前往后遍历的时候找到一个可以删除的立即删除，删除之后，继续遍历新的数组，继续删除。用一个栈模拟解密过程就行。

时间复杂度 $O(S1*S2)$

```
const int maxn=1000000+10;
char stack[maxn];
char s1[maxn],s2[100+10];
int top=0;
bool judge(int t){
    for(int i=t-1,j=top;i>=0;i--,j--){
        if(stack[j]!=s2[i]) return false;
    } return true;
}
int main()
{
    scanf("%s",s1);
    scanf("%s",s2);
    int t1=strlen(s1);
    int t=strlen(s2);
    for(int i=0;i<t1;i++){
        stack[++top]=s1[i];
        if(top>=t) if(judge(t)) top-=t;
    }
    for(int i=1;i<=top;i++) putchar(stack[i]);
    printf("\n");
    return 0;
}
```



栈模拟

如果每次操作都是在末尾进行，那么一般情况下都可以用栈来模拟，比如字符串模拟，括号匹配，当然也可能只是一道结论题

- 1: 洛谷 1739 表达式括号匹配
- 2: MSOJ 1187 括号匹配
- 3: LOJ 10150 括号配对
- 4: MSOJ 1270 小W的爱情密码
- 5: MSOJ 1388 字符串简化
- 6: POJ 1028 网站导航



单调栈

顾名思义，就是维护栈内的元素单调递增/递减，通过栈的单调性来完成一系列问题。他常用来优化某些问题的时间复杂度，达到优化效率的效果，常常与二分查找相结合。

单调栈经常用来优化某些算法：

- (1)对于每一个数，寻找他右边第一个比他大(小)的数
- (2)对于给定序列，寻找一子序列最小（大）值*序列长度(之和)之积最大(小)
- (3)维护并查询序列的单调性



→ 单调栈模板一

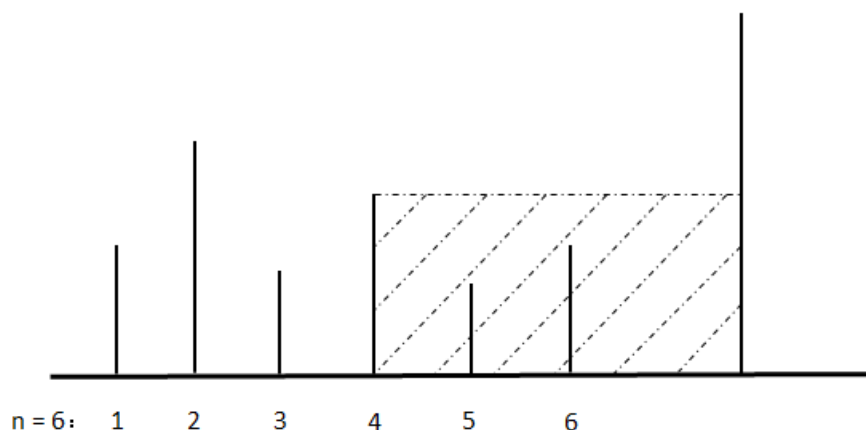
```
int top=0; //栈为空
for(int i=1;i<=n;i++)
{
    while(top!=0 && s[top]<a[i]) top--; //循环执行出栈操作
    s[++top]=a[i]; //入栈
}
```

因为对于当前这个数 $a[i]$ ，无论和栈顶元素关系如何，他都会入栈，所以我们可以先执行出栈操作，再执行入栈操作，这样就省掉了条件语句。



例题：木板倒水

地上从左到右竖立着 n 块木板，从 1 到 n 依次编号，如下图所示。我们知道每块木板的高度，在第 n 块木板右侧竖立着一块高度无限大的木板，现对每块木板依次做如下的操作：对于第 i 块木板，我们从其右侧开始倒水，直到水的高度等于第 i 块木板的高度，倒入的水会淹没 a_i 块木板（如果木板左右两侧水的高度大于等于木板高度即视为木板被淹没），求 n 次操作后，所有 a_i 的和是多少。如图上所示，在第 4 块木板右侧倒水，可以淹没第 5 块和第 6 块一共 2 块木板， $a_4 = 2$ 。（注意要，该块木板的左右两边都被水淹没才算是木板被淹没）



输入：

5

10 5 8 12 6

$n \leq 1000000$

输出：

3

木板倒水—问题分析

暴力做法：

这道题直接做很简单，遍历 n 根柱子，找到每根柱子右边第一个高度不小于他的柱子就表示寻找结束，中间的就是可以淹没的。尝试 每根柱子的时间复杂度是 $O(n)$ ，对于每一根柱子往右边依次寻找时间复杂度也是 $O(n)$ ，所以总的时间复杂度是 $O(n^2)$ ，然后数据范围是 $100w$ ，很明显，过不了。

问题优化：

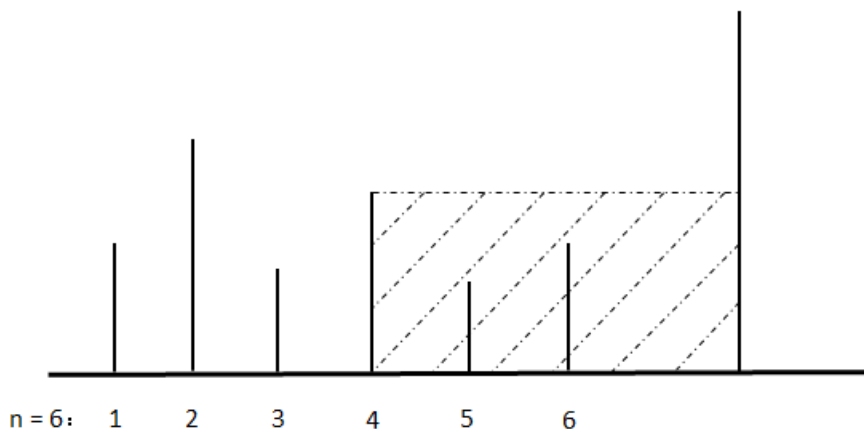
再仔细来分析一下这道题，对于第 i 根柱子，从右往左看，有两种情况：

(1)对于第 $i-1$ 根柱子，如果 $h[i-1] \leq h[i]$ ，则说明 $h[i]$ 及 i 之后的柱子均不能被淹没，则 $h[i-1]$ 入栈，下一个。

(2)对于第 $i-1$ 根柱子，如果 $h[i-1] > h[i]$ ，则说明存在淹没的情况，因此继续往后找，直到第 k 根柱子高度 $h[i-1] \leq h[k]$ ，则从 i 到 $k-1$ 之间的柱子都是可以淹没的，累加该段答案。然后 $h[i-1]$ 入栈。

→ 木板倒水—时间复杂度

我们来大概算一下时间复杂度，所有柱子均会入栈一次（入栈等待终止柱的到来），当遇到终止柱的时候出栈一次，最后栈里元素值成单调递减序列，所以叫单调递增栈。所以时间复杂度就是 $O(2*n)$ 。



```
int ans=0;
stack[++top]=INF;
for(int i=n;i>=1;i--){
    while(stack[top]<h[i]){
        ans++; top--;
    }
    stack[++top]=h[i];
}
```




NOIP2018初赛题目

- 对于每一个数，寻找他右边第一个比他大(小)的数，比如：
3 2 4 5 6 3 1，输出为：4 4 5 6 0 0 0
- 对于给定序列，寻找一子序列最小（大）值*序列长度（之和）之积最大(小)
- **给定n个正数，一个区间的权值定义为：
这个区间的所有数之和乘以这个区间内最小的数
求所有区间中权值最大的区间**



例题：最长上升子序列

最长上升子序列 (LIS)

设有整数序列 $b_1, b_2, b_3, \dots, b_m$ ，若存在下标 $i_1 < i_2 < i_3 < \dots < i_n$ ，且 $b_{i_1} < b_{i_2} < b_{i_3} < \dots < b_{i_n}$ ，则称 $b_1, b_2, b_3, \dots, b_m$ 中有长度为 n 的不下降序列 $b_{i_1}, b_{i_2}, b_{i_3}, \dots, b_{i_n}$ 。

求序列 $b_1, b_2, b_3, \dots, b_m$ 中所有长度 (n) 最大上升子序列

输入：整数序列。

输出：最大长度 n 和所有长度为 n 的序列个数。

范围： $M \leq 100000$

➔ 最长上升子序列—问题分析

问题分析：

1、可否贪心？贪心策略是什么？

2、暴力做法：通过对问题分析发现，最长上升子序列的结尾元素可以是任意一个，因此我们考虑如何构造以 $a[i]$ 结尾的最长上升子序列，以 $a[i]$ 结尾的最长上升子序列，肯定是从1到 $i-1$ 中比 $a[i]$ 小，且子序列长度最大的那个元素的长度+1构成。因此，假设我们用 $dp[i]$ 表示以 i 元素结尾的最长上升子序列。

则最后的答案： $ans = \max(dp[i]), 1 \leq i \leq n;$

$dp[i] = \max(dp[j] + 1), 1 \leq j \leq i-1 \text{ 且 } a[j] < a[i].$

但这样子做的是时间复杂度是 $O(n*n)$ 超时。

如何来优化时间复杂度了？

➔ 最长上升子序列—问题优化

由于以第 i 个数结尾时找的是由前 $i-1$ 个数中比第 i 个数小，且长度最长的那个转移过来。那么这个寻找过程是否可以优化呢？

初看是没办法优化的，但进一步思考分析，对于相同长度的上升子序列，我们肯定是希望序列末尾元素值越小越好，这样后面便可以加更多的元素，如果我们维护一个最优的序列呢？则对于第 i 个数是否有更快的处理方法？

➔ 最长上升子序列—问题优化

- 如果我们维护出一个相同长度，末尾元素最优的序列，则发现这个序列长度成单调性(长度依次增加啊),且序列元素的值也成单调性(随着长度增加，值肯定也是越来越大)。
- 因此，对于第*i*个元素我们便和序列末尾元素比较，如果比它大直接加入(说明构成一个更长的序列)；如果比它小，则由第*i*个元素去更新序列第*i*个数结尾中长度最长的最有序列(即长度最长以第*i*个数结尾的末尾元素最优)：如：

给定序列: 1 2 3 4 6 5 8 10 9

维护序列: 1 2 3 4 5 8 9

- 基于此，我们便二分动态维护这个序列，最后维护序列的长度就是我们最长上升子序列的长度。时间复杂度 $n \cdot \log(n)$
- 思考：维护的序列是不是一定是合法的最长上升子序列？

➔ 最长上升子序列—问题优化

- 基于此，我们便可以二分动态维护这个序列，最后维护序列的长度就是我们最长上升子序列的长度。时间复杂度 $n \times \log(n)$
- 二分STL函数：lower_bound和upper_bound,在从小到大的排序数组中，
lower_bound(begin,end,num): 从数组的begin位置到end-1位置二分查找第一个大于或等于num的数字，找到返回该数字的地址，不存在则返回end。通过返回的地址减去起始地址begin,得到找到数字在数组中的下标。
upper_bound(begin,end,num): 从数组的begin位置到end-1位置二分查找第一个大于num的数字，找到返回该数字的地址，不存在则返回end。通过返回的地址减去起始地址begin,得到找到数字在数组中的下标。
在从大到小的排序数组中，重载lower_bound()和upper_bound()
lower_bound(begin,end,num,greater<type>()): 从数组的begin位置到end-1位置二分查找第一个小于或等于num的数字，找到返回该数字的地址，不存在则返回end。通过返回的地址减去起始地址begin,得到找到数字在数组中的下标。
在上升子序列中用:lower_bound(因为要求严格递增)
在下降子序列中用upper_bound
- 二个函数常见用法: https://blog.csdn.net/qq_40160605/article/details/80150252



最长上升子序列—参考代码

```
int stack[maxn];
int n,top=0;
int a[maxn];
int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d",&a[i]);
    stack[++top]=a[1];
    for(int i=2;i<=n;i++){
        if(a[i]>stack[top]){
            stack[++top]=a[i];
        } else {
            int k=lower_bound(stack+1,stack+top+1,a[i])-(stack);
            stack[k]=a[i];
        }
    }
    printf("%d\n",top);
    for(int i=1;i<=top;i++) printf("%d ",stack[i]);
    printf("\n");
    return 0;
}
```


➔ 最长上升子序列—问题优化

针对第二问：求出最长上升子序列的个数，普通 $O(n*n)$ 的求法，比较容易，但是如果优化成 $n*\log n$ ，这需要用到高级数据结构，等我们以后再讲。

```
dp[1]=t[1]=1;
for(int i=2;i<=n;i++){
    for(int j=1;j<i;j++){
        if(a[i]>a[j]){
            if(dp[i]<dp[j]+1){
                dp[i]=dp[j]+1;t[i]=t[j];
            } else if(dp[i]==dp[j]+1){
                t[i]+=t[j];
            }
        }
    }
}
```




单调栈练习题

- 1: MSOJ 1131 木板倒水
- 2: MSOJ 1133 区间最大值
- 3: MSOJ 1136 最大长方形
- 4: 洛谷 1020 导弹拦截
- 5: 洛谷 1823 音乐会的等待
- 6: MSOJ 1184 最长不下降子序列
- 7: SCU 2511 Moooo
- 8: MSOJ 1344 最大矩形(二维单调栈)

→ 卡特兰数

卡特兰数又称卡塔兰数，**卡特兰数是组合数学中一个常出现在各种计数问题中的数列。**

卡特兰数是一种经典的组合数，其前几项为（从第零项开始）：1, 1, 2, 5, 14, 42, 132...他经常用于计算**有关出入栈的方法**这一类的题。他的计算公式为：

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

卡特兰数 C_n 满足以下递推关系：

$$1. C_{n+1} = C_0 C_n + C_1 C_{n-1} + \cdots + C_n C_0$$

$$2. (n-3) C_n = \frac{n}{2} (C_3 C_{n-1} + C_4 C_{n-2} + C_5 C_{n-3} + \cdots + C_{n-2} C_4 + C_{n-1} C_3)$$

→ 卡特兰数

设 $h(n)$ 为catalan数的第 $n+1$ 项，令 $h(0)=1, h(1)=1$ ，
catalan数满足递推式：

$$h(n) = h(0)*h(n-1) + h(1)*h(n-2) + \dots + h(n-1)*h(0) \quad (n \geq 2)$$

例如： $h(2) = h(0)*h(1) + h(1)*h(0) = 1*1 + 1*1 = 2$

$h(3) = h(0)*h(2) + h(1)*h(1) + h(2)*h(0) = 1*2 + 1*1 + 2*1 = 5$

另类递推式：

$$h(n) = h(n-1) * (4*n-2) / (n+1);$$

递推关系的解为：

$$h(n) = C(2n, n) / (n+1) \quad (n=0, 1, 2, \dots)$$

递推关系的另类解为：

$$h(n) = c(2n, n) - c(2n, n-1) \quad (n=0, 1, 2, \dots)$$



→ 卡特兰数的应用

题目大意：给定一个操作序列 $1, 2, 3 \dots n$ ，依次入栈，问可能的出栈序列有多少种($n \leq 1000$)，答案对 100007 取余

输入：
3

输出：

5

(1 2 3)

(1 3 2)

(2 1 3)

(2 3 1)

(3 2 1)



解题思路

拿到这道题的第一种思路就是爆搜模拟，尝试可能的出入栈操作，但是，很明显会T。对于这种题，数据较大的，不是结论题就是递推公式题。

我们先考虑能不能递推。也就是思考当前状态和前一状态的关系：

假设最后一个出栈的元素为 x ，那么对于 x 之前的元素 $(x-1)$ ，一定是比 x 先入栈并且先出栈的。对于 x 之后的元素 $(n-x)$ ，一定是比 x 后入栈并且先出栈的。并且相互之间是独立的两部分。

设 $f[i]$ 表示有 i 个数时的答案，最后一个出栈的数为 j ($1 \leq j \leq i$)，对于每一种情况 j ，方案数为 $f[j-1] * f[i-j]$ 。



解题思路

```
f[0] = 1;  
f[1] = 1;  
for(int i=2; i<=n; i++)  
{  
    for(int j=0; j<i; j++)  
        f[i] += f[j] * f[i-j-1];  
}
```

其实这就是卡特兰数的递推式。

什么时候可以用卡特兰数呢？

要选某个元素1，必须要保证元素1（包括元素1）前元素0的个数 $\geq$ 元素1的个数，最后1和0的个数必须一样。



→ 解题思路

1. 给出一个 n ，要求一个长度为 $2n$ 的01序列，使得序列的任意前缀中1的个数不少于0的个数，求方案数。
2. 所有不同构的含 n 个分枝结点的二叉树的个数。
3. 所有在 $n \times n$ 格点中不越过对角线的单调路径的个数。一个单调路径从格点左下角出发，在格点右上角结束，每一步均为向上或向右。
4. 通过连结顶点而将 $n + 2$ 边的凸多边形分成三角形的方法个数
5. n 个数依次进栈，可随机出栈，求方案数。
6. $P = a_1 \times a_2 \times a_3 \times \dots \times a_n$ ，依据乘法结合律，不改变其顺序，只用括号表示成对的乘积，问有几种括号化的方案。
7. 有 $2n$ 个人排成一行进入剧场。入场费5元。其中只有 n 个人有一张5元钞票，另外 n 人只有10元钞票，剧院无其它钞票，问有多少中方法使得只要有10元的人买票，售票处就有5元的钞票找零？

→ 队列 (Queue)

外特性：先进先出(FIFO)

- 食堂排队
- 吸管里的饮料



- 数组实现：元素`queue[maxn]`，队首`head`，队尾`tail`

- 入队：`queue[tail++] = element;`
- 出队：`element = queue[head++];`
- 队空条件：`head >= tail`
- 问题：出队的元素还在数组里，不是很浪费吗？

→ 队列 (Queue) STL

- STL (Standard Template Library)

```
#include <queue>
```

```
using namespace std;
```

```
queue<int> q;
```

```
int x=q.front();返回队首元素, 仅适用于FIFO队列
```

```
q.empty();
```

```
q.size();
```

```
q.pop();
```

```
int y=q.back();返回队尾元素, 仅适用于FIFO队列
```

```
q.top();仅适用于优先队列
```

```
q.push();
```



队列

- (1) 普通队列模拟
- (2) bfs搜索
- (3) 最短路
- (4) 双端队列
- (5) 单调队列优化
- (6) 优先队列模拟

→ 队列模拟--卡片游戏

桌面上有一叠 n 张牌，从上往下编号 $1 \sim n$

当至少还剩两张牌时进行以下操作：

把第一张牌扔掉，把新的第一张牌放到最后

输出扔掉牌的序列，以及最后剩下的一张牌

问题分析：

两种操作符合队列的特点

按照要求用队列模拟即可



→ 队列模拟

普通队列模拟的题相当于栈模拟来说很少，用途不是很广。
普通队列主要用于搜索和spfa

洛谷 P1540 机器翻译

洛谷 P1638 逛画展





→ 队列搜索

搜索：常见的搜索策略有两种：DFS和BFS

DFS：是一种基于深度的优先搜索，是一种按照深度穷举所有路径的搜索方法，常常用于暴力枚举，也称之为暴搜，再利用DFS做搜索题时除了理清搜索策略及思路外，更多的考虑搜索的不必要条件，即搜索剪枝。常见的剪枝有：最优性、可行性、重复性等三种剪枝方法。

BFS：是一种基于层次的优先搜索，也叫广度优先搜索，他是层次（离起点远近）穷举所有路径的搜索方法。常常用于求解从起点走到终点，**最少需要走几步这些情况（权值为1的最短路）这时BFS往往比DFS跟优。**BFS做题时常常需要考虑的重复状态的判断，这个比较难，状态判从我们常常用hash+链表的方式实现。



➔ 队列搜索--走迷宫

走迷宫

在一个 $x*y$ 的迷宫中，编程帮老鼠从左上角入口走到右下角出口，有些格子可能不能走

要求：找出最短路径长度



→ 八数码问题

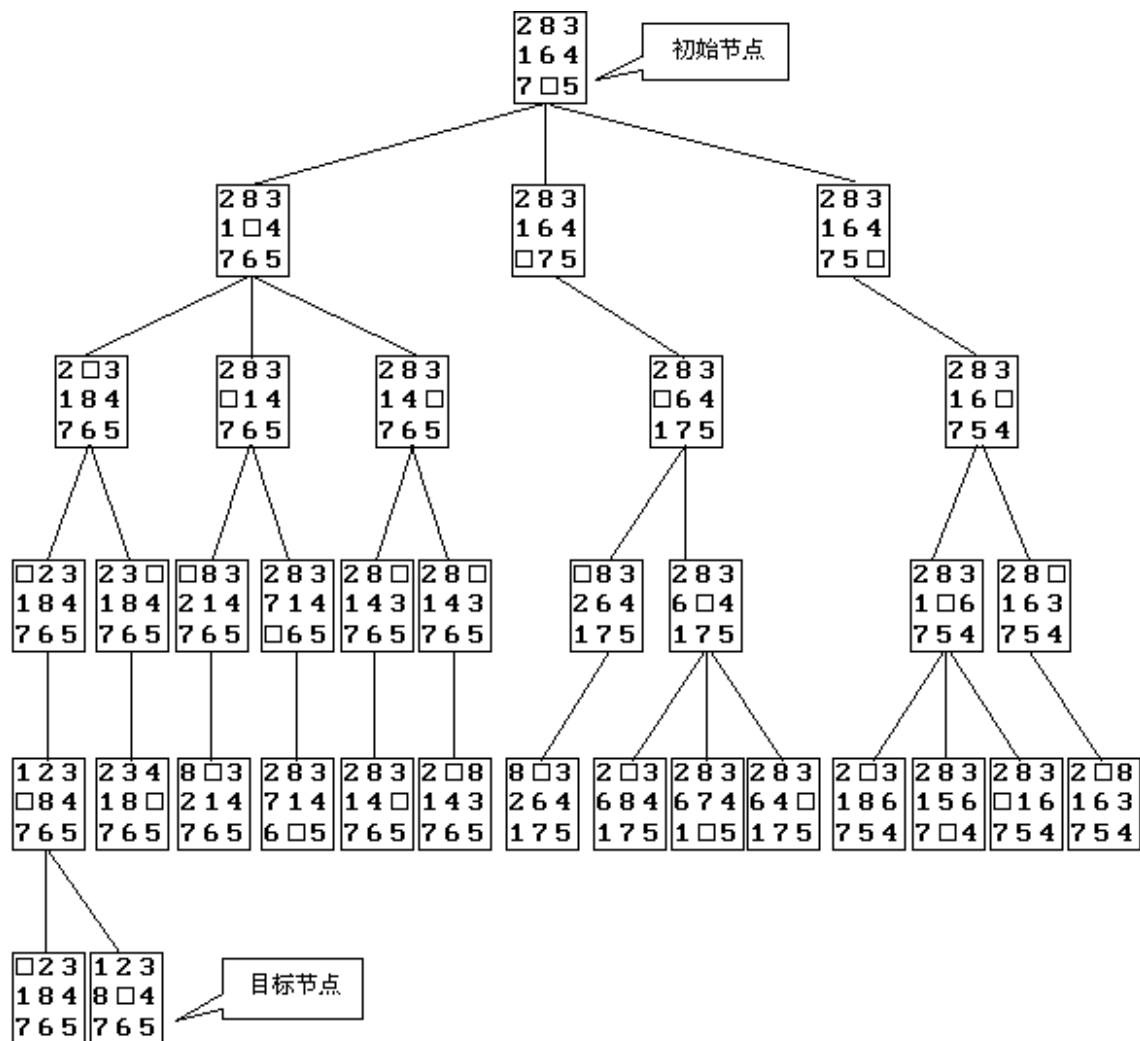
在 3×3 组成的九宫格棋盘上，摆有八个将牌，每一个将牌都刻有1—8中的某一个数码。棋盘中留有一个空格，允许其周围的某一个将牌向空格中移动，如右图所示。这样通过移动将牌就可以不断改变的布局结构，给出一个初始布局（称初始状态）和一个目标布局（称目标状态），问如何移动将牌，才能实现从初始状态到目标状态的转换。

2	8	3
1	6	4
7		5

1	2	3
8		4
7	6	5

八数码问题的初始和目标状态

八数码问题扩展过程





➔ 队列优化—最短路径

用队列优化dijkstra算法，就是我们常见的SPFA算法。





→ 单调队列

单调队列原理和单调栈相同，唯一不一样的地方在于单调栈中的元素在出栈之前都会对后面起作用，但是单调队列主要适用于前面的元素只在部分时间内起作用，时间一过都不再起作用了。**即单调队列主要用于区间问题**

单调队列操作和普通队列不太一样，可以在队首删除，可以在队尾插入和删除，所以最好用手写队列，不用STL队列。

旅行家的预算

- 一个旅行家想驾驶汽车以最少的费用从一个城市到另一个城市（假设出发时油箱是空的）。给定两个城市之间的距离 $D1$ 、汽车油箱的容量 C （以升为单位）、每升汽油能行驶的距离 $D2$ 、出发点每升汽油价格 P 和沿途油站数 N （ N 可以为零），油站 i 离出发点的距离 D_i 、每升汽油价格 P_i （ $i=1, 2, \dots, N$ ）。计算结果四舍五入至小数点后两位。如果无法到达目的地，则输出“No Solution”。
- $N \leq 10^5$

样例输入

275.6 11.9 27.4 2.8 2
102.0 2.9
220.0 2.2

样例输出

26.95



旅行家的预算—参考代码

```
list[n+1]=d1;
head=tail=0;
que_p[++tail]=p;que_c[tail]=c;//起点油箱加满, 入队(价格, 油量)
for(int i=1;i<=n+1;i++) { //枚举每一个加油站
    //第一: 计算从上一站开过来需要的费用
    cc=0;
    d=list[i]-list[i-1]; //算出两点之间的距离
    while(que_c[head]*d2<d){
        ans+=que_c[head]*que_p[head];
        d-=que_c[head]*d2;
        cc+=que_c[head];
        head++;
    }
    ans+=que_p[head]*d/d2;
    que_c[head]-=(d/d2);
    cc+=(d/d2);
    if(i==n+1) break;
    //第二: 从队尾开始用当前加油站的油去替换油箱里比当前价位高的油。
    while(que_p[tail]>op[i]){
        cc+=que_c[tail--];
    }
    que_p[++tail]=op[i];que_c[tail]=cc;
}
printf("%.2f\n",ans);
```

→ P1886 滑动窗口

现在有一堆数字共N个数字 ($N \leq 10^6$)，以及一个大小为k的窗口。现在这个从左边开始向右滑动，每次滑动一个单位，求出每次滑动后窗口中的最小值和最大值。

输入：

8 3
1 3 -1 -3 5 3 6 7

输出：

-1 -3 -3 -3 3 3
3 3 5 5 6 7

Window position	Minimum value	Maximum value
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

$N \leq 100000, K \leq 100000;$

→ 滑动窗口—问题分析

本题暴力做法很简单，枚举起点，然后找长度为K的区间，再求出最小值。超时，不能过100%的数据。

于是，我们思考这样一个问题，**对于第i位置的元素 $a[i]$ ，如果它下一个相邻元素 $a[i+1]$ ，存在 $a[i] \geq a[i+1]$ ，则从当前开始区间K，最小值只可能是 $a[i+1]$ ，不可能是 $a[i]$ ，则 $a[i]$ 变没有意义了；如果存在 $a[i] < a[i+1]$ ，则 $a[i]$ 和 $a[i+1]$ 都是有用的。**因此，我们可以用一个单调递增队列来维护元素值。假如我们先求最小值，保证队首就是最小值，满足滑动窗口长度时输出队首。

- ✓ 例如样例 1 3 -1 -3 5 3 6 7
- ✓ 第一步 1 先入队了
- ✓ 第二步 1 小于3 没问题 所以3接着入队了
- ✓ 第三步 1和3 都大于-1 所以1和3都弹出 -1入队 此时刚好满足长度为k ($k=3$) 的滑动窗口，输出队首 -1
- ✓ 第四步 -1 大于 -3 所以-1弹出 -3 入队 输出队首-3
- ✓ 第五步 -3 小于5 没问题 5入队 输出队首-3
- ✓ 第六步 5大于3 那5不论怎么样都不可能是最小值了，所以弹出5，3入队，输出队首-3
- ✓ 第六步 3小于6没问题 6入队，此时滑动窗口覆盖范围已经走到了(5,3,6) 所以要将-3 出队，输出队首3
- ✓ 第七步 6小于7没问题，7入队，输出队首3

→ 滑动窗口

```
for(int i=1;i<=n;i++){
    while(head<tail&&q[tail-1].x<=num[i]) tail--; //出队
    //如果不满足上面条件就不会有出队操作
    q[tail].x=num[i];
    q[tail].y=i;
    tail++; //当前元素入队
    while(q[head].y<=i-k) //当移动到i的位置的时候,
    //当前队首元素如果不可能和i同一个窗口。就出队
    {
        head++;
    }
    if(i>=k) //输出最大值
    printf("%d ", q[head].x);
}
```

这两段代码不能反过来，如果a1是最大值，到a3的时候就会输出，到a4的时候a1才会出队

最好不要直接把第一个让进队列，可能出现k=1的情况



子序列和最大

给定一个长度为 n 的序列，找出一段连续的长度不超过 M 的子序列，和最大。

问题分析

第一步，将连续和转化成前缀和之差

对于一个固定的 s_i ，求 $s(i-M) \sim s(i-1)$ 中的最小值
用单调递增队列维护

跳房子

- ◆ n 个格子，格子的位置为 x_i ，分数为 s_i ，至少获得 k 分，任意时刻都能停
- ◆ 每次向右弹跳固定的 d ，可以修改灵活度 g ($d - g \sim d + g$)，求 g 的最小值
- ◆ ($n \leq 500000$, $d \leq 2000$, $x_i \leq 10^9$)

- ✓ 分析：
- ✓ $dp[i]$ 表示跳到第 i 个格子时获得的最大得分
- ✓ 枚举答案，直接 dp 判断 20分
- ✓ 二分答案，直接 dp 判断 50分
- ✓ 二分答案， dp 判断时用上单调队列，100分



→ 单调队列练习题

P1440: 求m区间内的最小值

P2032: 扫描

P1631: 序列合并

P2216: 理想正方形(二维单调队列)

P1725: 琪露诺 (简单的单调(优先)队列优化dp)

JOYOI: 最大子序和(单调队列优化DP)



优先队列

优先队列都是用STL的堆实现，有大根堆和小根堆两种，需要自己重载小于运算符。值得注意的是默认的优先队列是大根堆，即堆顶元素是最大的，所以如果我们是大根堆，重载小于运算符的时候还是小于，如果是小根堆，重载小于运算符的时候就要变成大于。一定不能错了



中位数

题目描述

给出一个长度为 N 的非负整数序列 A_i ，对于所有 $1 \leq k \leq (N + 1)/2$ ，输出 $A_1, A_3, \dots, A_{2k-1}$ 的中位数。即前 $1, 3, 5, \dots$ 个数的中位数。

输入输出格式

输入格式：

第1行为一个正整数 N ，表示了序列长度。

第2行包含 N 个非负整数 A_i ($A_i \leq 10^9$)。

输出格式：

共 $(N + 1)/2$ 行，第 i 行为 $A_1, A_3, \dots, A_{2i-1}$ 的中位数。

输入输出样例

输入样例#1： [复制](#)

```
7
1 3 5 7 9 11 6
```

输出样例#1： [复制](#)

```
1
3
5
6
```

➔ 中位数

- 可以考虑线段树来求解区间的中位数，但是比较麻烦，不太好找。我们可以考虑优先队列
- 开两个堆，一个小根堆，一个大根堆，始终保证小根堆的数 $\leq$ 大根堆的数，并且小根堆的数量=大根堆 (+1)
- 那么这样我们就可以发现，小根堆堆顶的数就是中位数，因为如果一共有 $(2n+1)$ 个数。小根堆有 $n+1$ 个数，除去堆顶的元素还有 n 个，大根堆有 n 个数，并且满足小根堆 $\leq$ 大根堆，那么该数就是中位数
- 每次新插入一个数，只需要维护两个堆的性质不变就可以了，只需要将堆顶元素和该数比较然后出入队就可以了



中位数

```
if(a[i]>q1.top().num)//当前数比队顶元素大，放入小根堆
{
    node2 t;
    t.num=a[i];
    q2.push(t);
    if(q2.size()>q1.size())//保证中位数在大根堆堆顶
    {
        int tmp=q2.top().num;//取出小根堆的堆顶放入大根堆
        q2.pop();
        node1 t2;
        t2.num=tmp;
        q1.push(t2);
    }
}
else//如果小于则放在大根堆
{
    node1 t;
    t.num=a[i];
    q1.push(t);
    if(q1.size()-q2.size()>1)//大根堆数太多了
    {
        int tmp=q1.top().num;
        q1.pop();
        node2 t2;
        t2.num=tmp;
        q2.push(t2);
    }
}
```



优先队列练习题

- 1: 洛谷 1168 中位数
- 2: HDU 1896 踢石头
- 3: 洛谷 1801 黑匣子
- 4: 洛谷 4053 建筑抢修(难题)
- 5: 洛谷 2278 操作系统

分块—优雅的暴力

前言：

- 首先，我们来考虑这样一个模型：
- **有一段连续的序列 $a[1] \sim a[n]$** ，然后需要完成几类操作：
- **出题人：求出其中一段区间的和**

智商180的某宝宝：哎呀，你怎么这么傻，直接记录这个序列的前缀和不就得了？记录 $a[1] \sim a[i]$ 的和为 $sum[i]$ ，然后显然有 $sum[i+1] = sum[i] + a[i+1]$ ，我们要求 $a[l] \sim a[r]$ 就直接 $sum[r] - sum[l-1]$ 。

- **出题人：区间加上某个值**

高二大佬，两分钟后：我会一种叫线段树的东西(一种树形结构，可以维护区间求和及区间修改的优秀数据结构)！！！！

- **出题人：查询一段区间上有多少个数 $< k$ ($k > 0$ 且给定)**
- **出题人：**对了，忘了告诉你了， k 每次都不一样，还有，极其的多。另外，为了防止装*不让写平衡树(另一种能干很多事情的优秀数据结构)+线段树，占用空间不能超过****Mb

➔ 分块—优雅的暴力

- ◆ **分块**，顾名思义，就是把一段序列分成一小块一小块的来处理，维护。我们把一段当成一个整体，只记录维护整体的有关信息，就是分块。
- ◆ 分块对很多区间问题，是一种既简单又好用处理方式。
- ◆ 首先，对于前言说得那道题，很朴素的做法就是：
 1. 从询问的l到r扫过去，每次加上扫到的值,即 $ans = \sum_{l}^r a[i]$
 2. 直接把a[i]重新赋值不就得了 $a[i] = newa[i]$;
 3. 从询问区间的l到r扫过去,每回遇到 $< k$ 的位置，答案+1
- ◆ 分块就是对上述这个基础方法上暴力优化的！！！！

分块—优雅的暴力

分块的思想：假设我们总共的序列长度为 n ，然后我们把它切成 $\sqrt{n}$ 块，然后把每一块里的东西当成一个整体来看。然后对一个完整的块，整体处理；对于一个非完整的块，暴力维护。

分块的几个术语：

完整块：被操作区间完全覆盖的块

不完整块：操作区间不完全覆盖的块

对于前言中的第二个问题：

首先预处理每一完整块的和 $sum[i]$ ， i 表示第 i 块。

然后对于每次加上的数 x ，我们用一个 $lazy$ （为什么用 $lazy$ ，因为我懒）标记，来表示整个块被加上过多少了。

对于完整块： $lazy +=$ 加上的数 x ；块内的和 $sum[i] += x * \text{元素个数}$ （因为每个元素都被加上了 x ）。

对于不完整块：直接暴力枚举修改，然后跟新 $sum[i] += x * \text{元素个数}$ 。



➔ 分块—优雅的暴力

- ◆ 分块的效率要低于树状数组和线段树，但代码实现比较随意，相对来说更好理解。俗话说的好：越简单的东西，就意味着它越牢固，可拓展性越强。分块与其说是数据结构，它更像是一种思想，就像分治这样，虽然分块的整体框架也基本固定。分块与树状数组，线段树相比，它最大的优点在于，它本身极为强大的可拓展性。也就是说**分块可以容纳相当多的区间操作，这也是由分块本身的性质决定的，分块更多的像是一个思想。**
- ◆ 分块实现基本框架：1.划分块、2.预处理、3.操作或查询
操作或查询通常为4步：
 - 1.判断要操作或是查询的区间是否在一个块内
 - 2.若在一个块内，暴力操作或查询
 - 3.若不在一个块内，将除了最左边和最右边这两个块外其余的块进行整体的操作，即直接对块打上修改标记之类的
 - 4.单独暴力处理最左边的块和最右边的块

分块—优雅的暴力

分块操作：

第一步：分成 $\sqrt{n}$ 块，并预处理

```
n = read(); t = sqrt(n);
for(int i=1;i<=n;++i) a[i]=read();//读入
for(int i=1;i<=t;++i) {
    L[i]=(i-1)*t+1;//L[i]表示第i块的左端点
    R[i]= i*t;//R[i]表示第i块的右端点
}
if(R[t]<n) t++,L[t]=R[t-1]+1,R[t]=n;
for(int i=1;i<=t;++i)
for(int j=L[i];j<=R[i];++j) {
    pos[j]=i;//pos[j]=i表示第j个元素属于第i块
    sum[i]+=a[j];
}
```

➔ 分块—优雅的暴力

分块操作:

第二步: 修改操作

```
inline void change(int l, int r, int d) {
    int p=pos[l], q=pos[r];
    if(p==q) {//表示l~r在同一块了, 暴力处理
        for(int i=l; i<=r; ++i) a[i]+=d;
        sum[p]+=(r-l+1)*d;
    }
    else {
        for(int i=p+1; i<=q-1; ++i) add[i]+=d;//处理完整块
        for(int i=l; i<=R[p]; ++i) a[i]+=d;//暴力处理非完整块前面部分
        sum[p]+=(R[p]-l+1)*d;
        for(int i=L[q]; i<=r; ++i) a[i]+=d;//暴力处理非完整块后面部分
        sum[q]+=(r-L[q]+1)*d;
    }
}
```

分块—优雅的暴力

分块操作：第三步：查询统计答案

```
inline ll ask(int l, int r, int mod) {
    int p=pos[l], q=pos[r];
    ll ans=0;
    if(p==q) {
        for(int i=l; i<=r; ++i) ans=(ans+a[i])%mod; //同一段, 暴力加答案
        ans=(ans+add[p]*(r-l+1))%mod; //加上标记上的值
    }
    else {
        for(int i=p+1; i<=q-1; ++i) //完整块处理
            ans=(ans+sum[i]+add[i]*(R[i]-L[i]+1))%mod;
        for(int i=l; i<=R[p]; ++i) ans=(ans+a[i])%mod;
        ans=(ans+add[p]*(R[p]-l+1))%mod;
        for(int i=L[q]; i<=r; ++i) ans=(ans+a[i])%mod;
        ans=(ans+add[q]*(r-L[q]+1))%mod;
    }
    return ans%mod;
}
```




➡ 分块—优雅的暴力

分块入门的九大操作模型应用

➤ 分块入门 1：

给一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，单点查值。

➤ 问题分析：

我们给每个块设置一个加法标记(就是记录这个块中元素一起加了多少)，每次操作对每个整块直接 $O(1)$ 标记，而不完整的块由于元素比较少，暴力修改元素的值。

每次询问时返回元素的值加上其所在块的加法标记。

这样每次操作的复杂度是 $O(\sqrt{n}) + O(\sqrt{n})$

➡ 分块—优雅的暴力

➤ 分块入门 2：

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，询问区间内小于某个值 x 的元素个数。

➤ 问题分析：

有了上一题的经验，我们可以发现，数列简单分块问题实际上有三项东西要我们思考：

对于每次区间操作：

1. 不完整的块的 $O(\sqrt{n})$ 个元素怎么处理？
2. $O(\sqrt{n})$ 个整块怎么处理？
3. 要预处理什么信息（复杂度不能超过后面的操作）？

➔ 分块—优雅的暴力

➤ 区间询问操作：

对于不完整的块枚举统计即可；

对于完整块内寻找小于一个值的元素数，我们不得不要求块内元素是有序的，这样就能使用二分法对块内查询，需要预处理时每块做一遍排序，复杂度 $O(n\log n)$ 。每次查询在 $\sqrt{n}$ 个块内二分，以及暴力 $2\sqrt{n}$ 个元素，总复杂度 $O(n\log n + n\sqrt{n}\log\sqrt{n})$ 。

➤ 区间加操作：

利用上一题方法，维护一个加法标记，略有区别的地方在于，不完整的块修改后可能会使得该块内数字乱序，所以头尾两个不完整块需要重新排序。

➤ 加法标记下的询问操作：

块外还是暴力，查询小于 $(x - \text{加法标记})$ 的元素个数
块内用 $(x - \text{加法标记})$ 作为二分的值即可。

➔ 分块—优雅的暴力

➤ 分块入门 3:

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，询问区间内小于某个值 x 的前驱（比其小的最大元素）。

➤ 问题分析:

对于 $n \leq 100000$ 是为了区分暴力和一些常数较大的写法。接着第二题的解法，其实只要把块内查询的二分稍作修改即可。

不过这题其实想表达：可以在块内维护其它结构使其更具有拓展性，比如放一个set，这样如果还有插入、删除元素的操作，会更加的方便。



➡ 分块—优雅的暴力

➤ 分块入门4:

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，区间求和。

➤ 问题分析:

这题的询问变成了区间上的询问，不完整的块还是暴力；而要想快速统计完整块的答案，需要维护每个块的元素和，先要预处理一下。

考虑区间修改操作，不完整的块直接改，顺便更新块的元素和；完整的块类似之前标记的做法，直接根据块的元素和所加的值计算元素和的增量。

➡ 分块—优雅的暴力

➤ 分块入门5：

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间开方，区间求和

➤ 问题分析：

稍作思考可以发现，开方操作比较棘手，主要是对于整块开方时，必须要知道每一个元素，才能知道他们开方后的和，因此，难以快速对一个块信息进行更新。

看来要另辟蹊径。由于这题的修改就只有下取整开方，而一个数经过几次开方之后，它的值就会变成0或者1。

如果每次区间开方只不涉及完整的块，意味着不超过 $2\sqrt{n}$ 个元素，直接暴力即可。

如果涉及一些完整的块，这些块经过几次操作后就会都变成0/1，于是我们采取一种分块优化的暴力做法，只要每个整块暴力开方后，记录一下元素是否都变成了0/1，区间修改时跳过那些全为0/1的块即可。这样每个元素至多被开方不超过4次，显然复杂度没有问题。

分块—优雅的暴力

➤ 分块入门 6

给一个长为 n 的数列，以及 n 个操作，操作涉及单点插入，单点询问，数据随机生成。

➤ 问题分析

先说随机数据的情况

之前提到过，如果我们块内用数组以外的数据结构，能够支持其它不一样的操作，比如此题每块内可以放一个动态的数组，每次插入时先找到位置所在的块，再暴力插入，把块内的其它元素直接向后移动一位，当然用链表也是可以的。

查询的时候类似，复杂度分析略。

但是这样做有个问题，如果数据不随机怎么办？

如果现在一个块有大量单点插入，这个块的大小会大大超过 $\sqrt{n}$ ，那块内的暴力就没有复杂度保证了。

还需要引入一个操作：重新分块（重构）

每根号 n 次插入后，重新把数列平均分一下块，重构需要的复杂度为 $O(n)$ ，重构的次数为 $\sqrt{n}$ ，所以重构的复杂度没有问题，而且保证了每个块的大小相对均衡。当然，也可以当某个块过大时重构，或者只把这个块分成两半。

➡ 分块—优雅的暴力

➤ 分块入门 7:

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间乘法，区间加法，单点询问。

➤ 问题分析:

显然，如果只有区间乘法，和分块入门1的做法没有本质区别，但思考如何同时维护两种标记。

我们让乘法标记的优先级高于加法（如果反过来的话，新的加法标记无法处理）

若当前的一个块乘以 m_1 后加上 a_1 ，这时进行一个乘 m_2 的操作，则原来的标记变成 $m_1 * m_2$ ， $a_1 * m_2$

若当前的一个块乘以 m_1 后加上 a_1 ，这时进行一个加 a_2 的操作，则原来的标记变成 m_1 ， $a_1 + a_2$



➔ 分块—优雅的暴力

➤ 分块入门 8 :

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间询问等于一个数 c 的元素，并将这个区间的所有元素改为 c

➤ 问题分析：

区间修改没有什么难度，这题难在区间查询比较奇怪，因为权值种类比较多，似乎没有什么好的维护方法。

模拟一些数据可以发现，询问后一整段都会被修改，几次询问后数列可能只剩下几段不同的区间了。

我们思考这样一个暴力，还是分块，维护每个分块是否只有一种权值，区间操作的时候，对于同权值的一个块就 $O(1)$ 统计答案，否则暴力统计答案，并修改标记，不完整的块也暴力。

这样看似最差情况每次都会耗费 $O(n)$ 的时间，但其实可以这样分析：

假设初始序列是同一个值，那么查询是 $O(\sqrt{n})$ ，如果这时进行一个区间操作，它最多破坏首尾2个块的标记，所以只能使后面的询问至多多2个块的暴力时间，所以均摊每次操作复杂度还是 $O(\sqrt{n})$ 。

换句话说，要想让一个操作耗费 $O(n)$ 的时间，要先花费 $\sqrt{n}$ 个操作对数列进行修改。初始序列不同值，经过类似分析后，就可以放心的暴力啦。

➡ 分块—优雅的暴力

➤ 分块入门 9:

给出一个长为 n 的数列，以及 n 个操作，操作涉及询问区间的最小众数。

➤ 传送门:

这是一道经典难题，其实可以支持修改操作，具体见陈立杰大神的区间众数解题报告。

而且不强制在线的话有很多做法，可以看我blog一道类似题目 [czy的后宫3](#)

bzoj2724 是道强制在线区间众数，而且题目背景写的不错，这道题的题解就贴传送门咯

[\[bzoj2724\] \[Violet 6\]蒲公英](#)

➔ 分块—优雅的暴力

➤ 练习题目:

Luogu P2801 教主的魔法

BZOJ 2002:弹飞绵羊

BZOJ 2120: 数颜色

LuoguP4168 [Violet]蒲公英

➤ 参考学习博客:

数列分块入门1 – 9 by hzwer: <http://hzwer.com/8053.html>



四、哈希表

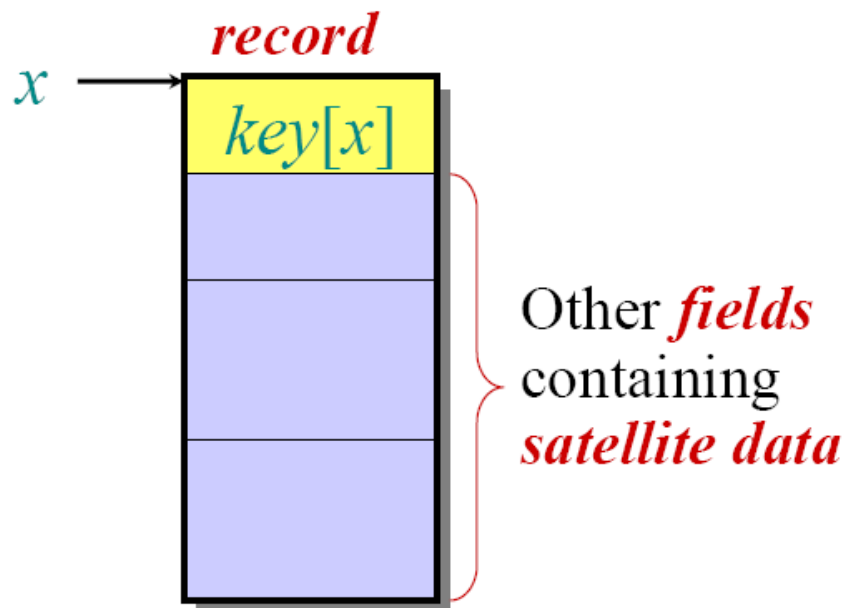




哈希表

- 哈希表(Hash table)经常被用来做字典(dictionary), 或称符号表(symbol-table)

Symbol table T holding n *records*:



Operations on T :

- INSERT(T, x)
- DELETE(T, x)
- SEARCH(T, k)

How should the data structure T be organized?

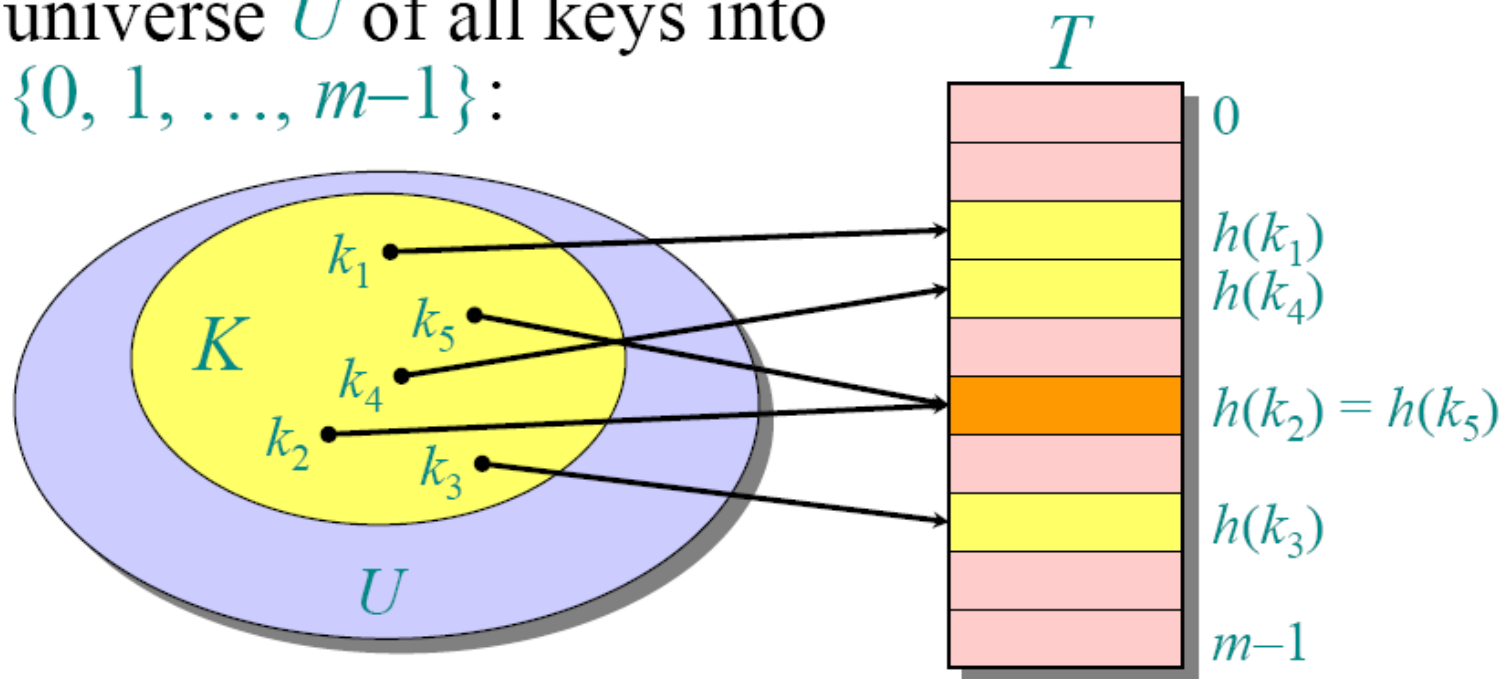


直接存取表

- 直接存取表(Direct-access table)的基本思想是: 如果key的范围为 $0 \sim m-1$ 而且所有key都不相同, 那么可以设计一个数组 $T[0..m-1]$, 让 $T[k]$ 存放key为 k 的元素, 否则为空(NIL)
- 显然, 所有操作都是 $O(1)$ 的
- **问题:** key的范围可能很大! 64位整数有18,446,744,073,709,551,616种可能, 而字符串的种类将会更多!

→ 解决方案: 哈希函数

Solution: Use a *hash function* h to map the universe U of all keys into $\{0, 1, \dots, m-1\}$:



When a record to be inserted maps to an already occupied slot in T , a *collision* occurs.



哈希函数的选取

- 严格均匀分布的哈希函数很难寻找, 但一般说来有三种方法在实际使用中效果不错
 - **取余法:** 取 $h(k) = k \bmod m$
 - **乘积法:** 取 $h(k) = (A * k \bmod 2^w) \text{ rsh } (w-r)$
 - **点积法:** 随机向量 a 和 k 的 m 进制向量做点积
- 实践中经常采用取余法



取余法

- 一般来说不要取 $m=2^r$, 因为它只取决于 k 的后 r 位

If $k = 1011000111\underbrace{011010}_2$ and $r = 6$, then
 $h(k) = 011010_2$. $h(k)$

- 一般取 m 为不太接近2或10的幂

NOI

→ 乘积法

- 取 $m=2^r$, 计算机字长为 w , 则

$$H(k) = (A * k \bmod 2^w) \text{ rsh } (w-r)$$

- 其中rsh表示右移位, A 是满足 $2^{w-1} < A < 2^w$ 的奇数. 注意: A 不应该太接近于 2^w . 由于乘法并对 2^w 取余是很快(自然会丢弃高位), 而算术右移也很快, 因此函数计算开销小



➔ 冲突解决

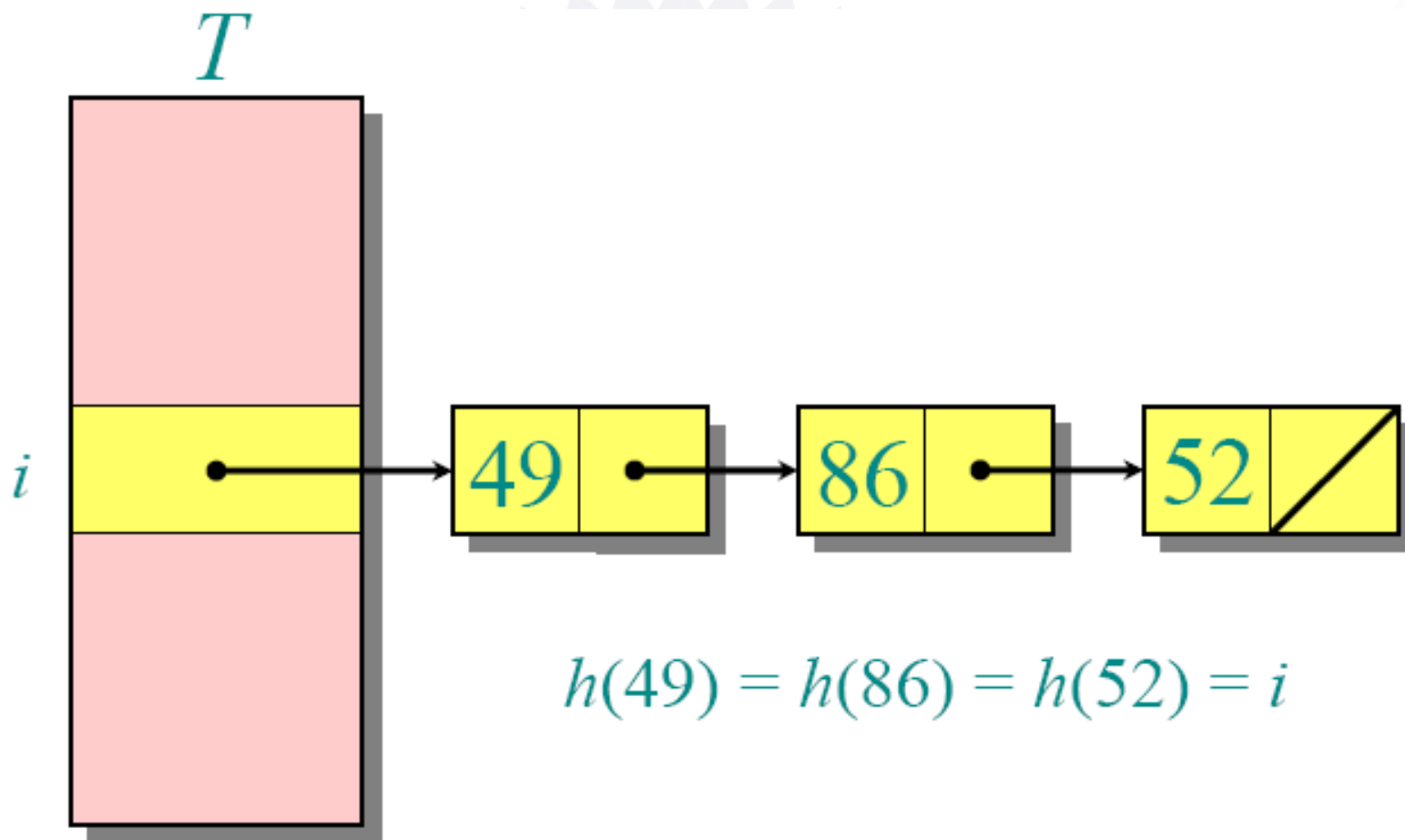
- 不同的key映射到同一个数, 称为冲突(collision), 冲突的两个元素显然不可能放在哈希表的同一个位置
- 通常有两种冲突解决方案(resolving collisions)
 - **链方法(chaining)**: 把key相同的串成链表
 - **开放地址法(open addressing)**: 自己的位置被占了, 就去占别人的





链地址法

- Key相同的元素形成一个链表



链地址法的查找效率

- 链地址法的时间效率取决于hash函数的分布. 我们假设每个k将等可能的被映射到任意一个slot, 不管其他key被映射到什么地方
- 设n为key的数目, m为不同的slot数, 装载因子 $\alpha = n/m$, 即每个slot平均的key数, 则

Expected time to search for a record with a given key = $\Theta(1 + \alpha)$.

*apply hash
function and
access slot*

*search
the list*

- $n=O(m)$ 时期望搜索时间是 $O(1)$ 的



开放地址法

- 开放地址法只使用表内的空间. 如果冲突产生, 计算出第二个可能的位置, 如果那里也有其他元素, 再看第三个可能的位置...即按一个探测序列查找
- 位置应该是key和探测的次数(probe number)的函数, 第i次探测位置为 $\text{slot} = h(k, i)$
- 每个slot都应能被探测到, 因此每个k的探测序列 $\langle h(k, 0), h(k, 1), h(k, 2), \dots, h(k, m-1) \rangle$
- 都应是 $\{0, 1, \dots, m-1\}$ 的排列
- 注意: 开放地址法不容易删除元素

开放地址法示例

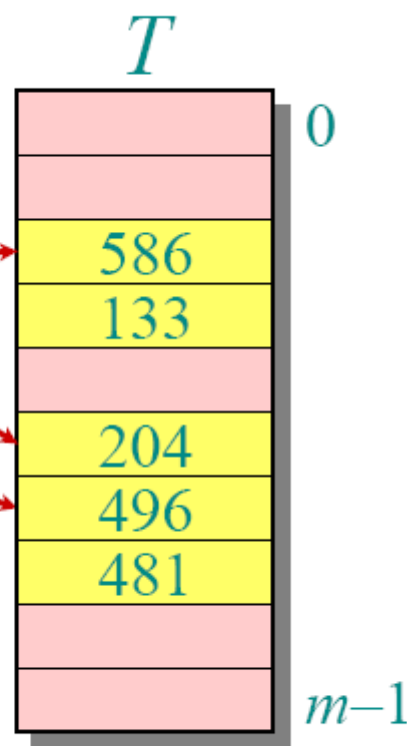
Search for key $k = 496$:

0. Probe $h(496,0)$

1. Probe $h(496,1)$

2. Probe $h(496,2)$

Search uses the same probe sequence, terminating successfully if it finds the key and unsuccessfully if it encounters an empty slot.





探测方法

- 线性探测:

$$h(k,i) = (h'(k) + i) \bmod m.$$

- 虽然很简单, 但是容易形成元素堆积
- 二次哈希: 组合两个哈希函数

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m.$$

- 一般效果会好很多, 但应保证 $h_2(k)$ 和 m 互素, 比如取 m 为2的幂而让 $h_2(k)$ 只产生奇数



开放地址法的查找效率

- 假设每个key等概率的把 $m!$ 个排列中的任何一个作为它的探测序列, 则有
- **定理:** 装载因子 $\alpha < 1$ 时, 不成功查找的期望探测次数为 $1/(1-\alpha)$
- 证明: 第 i 次探测到非空slot的概率为 $(n-i)/(m-i) < n/m = \alpha$
- 下面各种情况按概率加权计算期望

开放地址法的查找效率

- 各种情况按概率加权, 得期望的探测次数为

$$1 + \frac{n}{m} \left(1 + \frac{n-1}{m-1} \left(1 + \frac{n-2}{m-2} \left(\dots \left(1 + \frac{1}{m-n+1} \right) \dots \right) \right) \right)$$

$$\leq 1 + \alpha (1 + \alpha (1 + \alpha (\dots (1 + \alpha) \dots)))$$

$$\leq 1 + \alpha + \alpha^2 + \alpha^3 + \dots$$

$$= \sum_{i=0}^{\infty} \alpha^i$$

$$= \frac{1}{1 - \alpha} . \quad \square$$

The textbook has a more rigorous proof.



→ 查找效率比较

- 把开放地址法说通俗一点, 就是
 - 装载因子为常数时, 期望查找次数为常数
 - 一半装满时, 期望查找次数为 $1/(1-0.5)=2$
 - 装满90%时, 期望查找次数为 $1/(1-0.9)=10$
- 装得比较满(尤其是 $n > m$)时, 使用链地址法
 - 在哈希表里保存每个key的第一个元素 $\text{first}[0..m-1]$,
 - 在一个数组 $\text{data}[1..n]$ 里装着所有元素和下一个元素 $\text{next}[1..n]$

➔ 链地址法参考代码

- 在实际应用中，推荐使用链地址法
 - first[i]表示**哈希函数值为i**的第一个数据下标
 - key[i]和next[i]表示**第i个**数据的key和下一个

```
int find(int k){
    int h = hash(k);
    int p = first[h];
    while (p){ if(key[p] == k) return p; p = next [p]; }
    return 0;
}
void insert(int x){
    int h = hash(key[x]); next[x] = first[h]; first[h] = x;
}
```

The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two fluctuating lines is visible on the right side.

Thank you!